Prolog Tutorial and Resolution Prover Demo

Video Lesson

Duration: 00:14:28

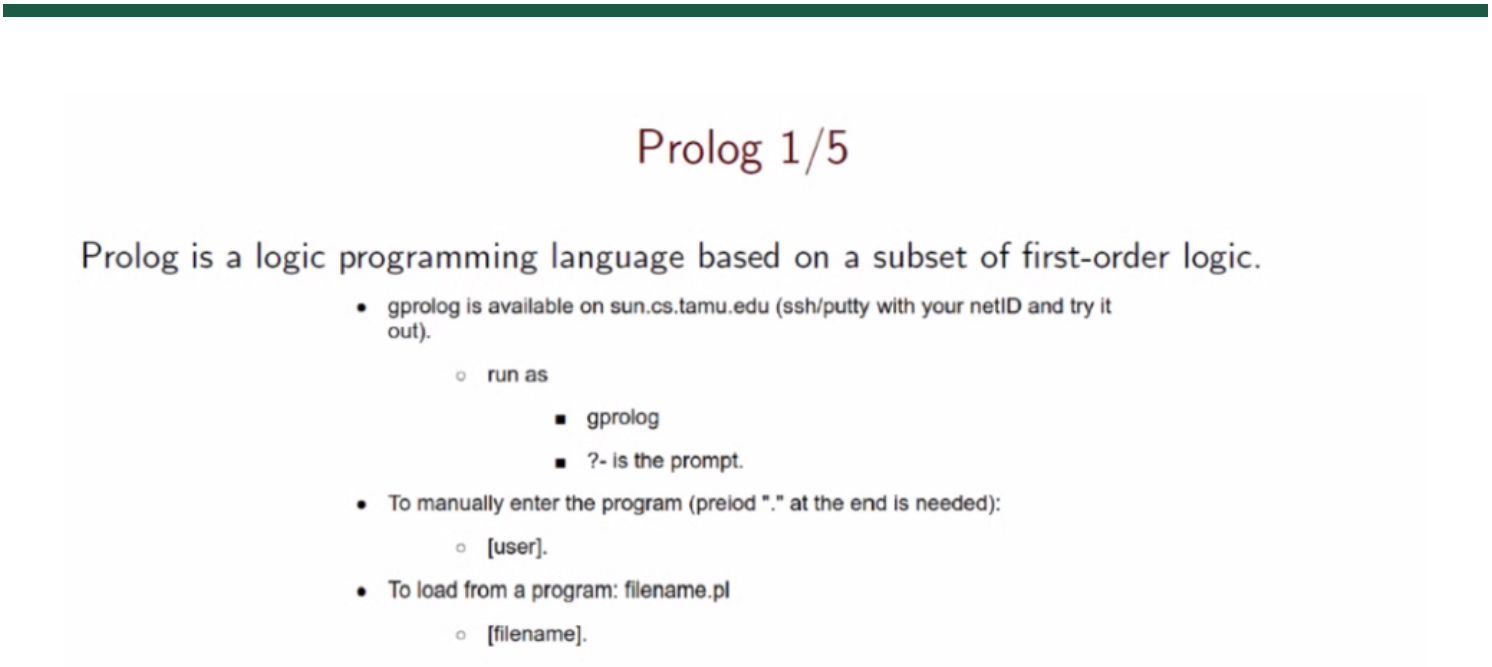
Slide Deck

M09S10 (https://canvas.tamu.edu/courses/430127/files/92570316?wrap=1)

Lesson Transcript

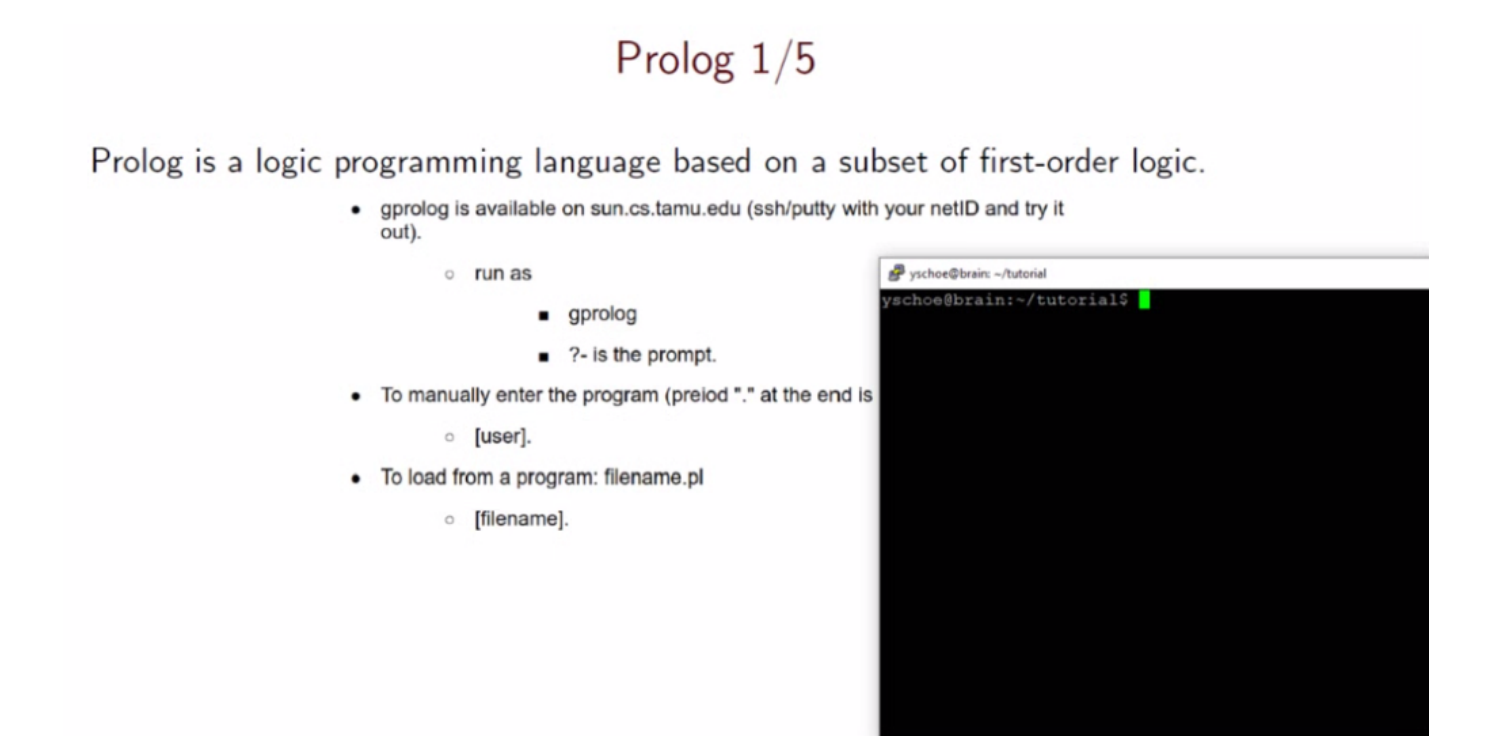
In this final lecture in this module, we will go through a tutorial on Prolog, a logic-based language,

and a resolution prover demo.



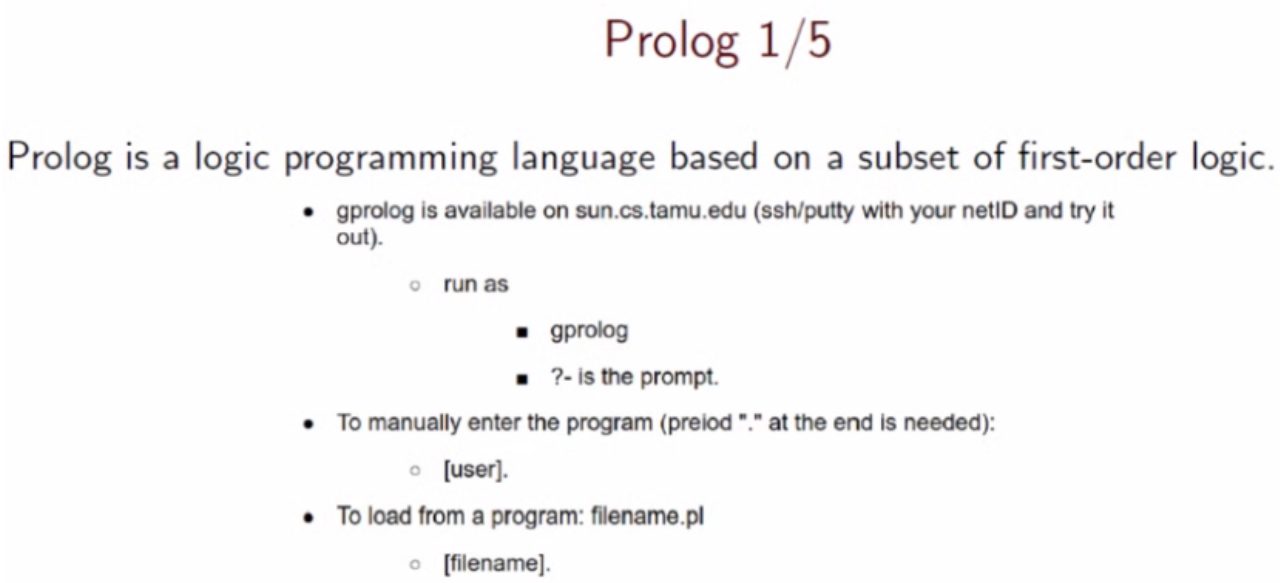
Prolog is a logic programming language based on a subset of first-order logic.

An open-source version of Prolog called gprolog is available on sun.cs.tamu.edu. We can either use a secure shell or putty to connect with netID and then try this out. We can run it as gprolog, and we will be given this prompt, question mark minus, where we can enter the commands.



Here is my Linux desktop, where I have gprolog installed. I just run gprolog, and we get the prompt. From here, we can enter the commands. To enter facts and rules, we can use this command. And always end it with this period. And we can enter facts and rules—for example, man(socrates).

And always end it with a period. And mortal(socrates), and so on. Then when we are done, we can press control-D, and we can check who is a man by issuing this query, man(X), where X is a variable. And this will give us the instance where man(X) is satisfied, and that is Socrates.



Manual entering all of these facts and rules using this user command is tedious. Instead, we can save all of these in a file and then load the file entirely using this command. The final name should have the extension .pl.

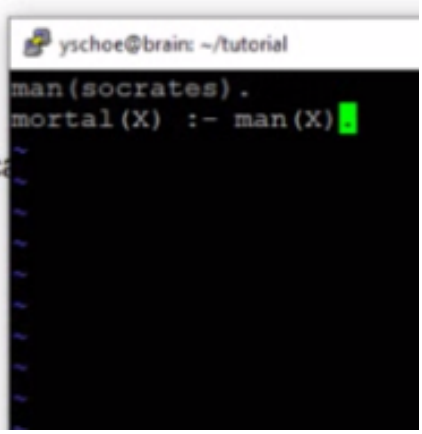
Prolog 2/5

- Simple program constructs:
  - program consitsts of
    - facts
    - rules
  - predicates, functions, and constants: lowercase
  - variables: uppercase
  - ":-" means "if" (to define a rule).
  - "," means "and"

The program in Prolog includes facts and rules. Predicates, functions, and constants start in lower case, variables start in upper case, and ":-" means if, and "," means end.

Prolog 2/5

- Simple program constructs:
  - program consitsts of
    - facts
    - rules
  - predicates, functions, and constants: lowerca
  - variables: uppercase
  - ":-" means "if" (to define a rule).
  - "," means "and"



So, here is a simple program containing one fact: Socrates is a man, and X is mortal if X is a man.

We save that file as socrates.pl, and we start gprolog.

Then, by issuing this command, socrates, we can load the program, and we can add in queries: man(X) to ask who is a man, and it gives us socrates. Then we can also ask questions: is Socrates mortal?

Then the answer is yes.

And once we are done, just press control-D to exit.

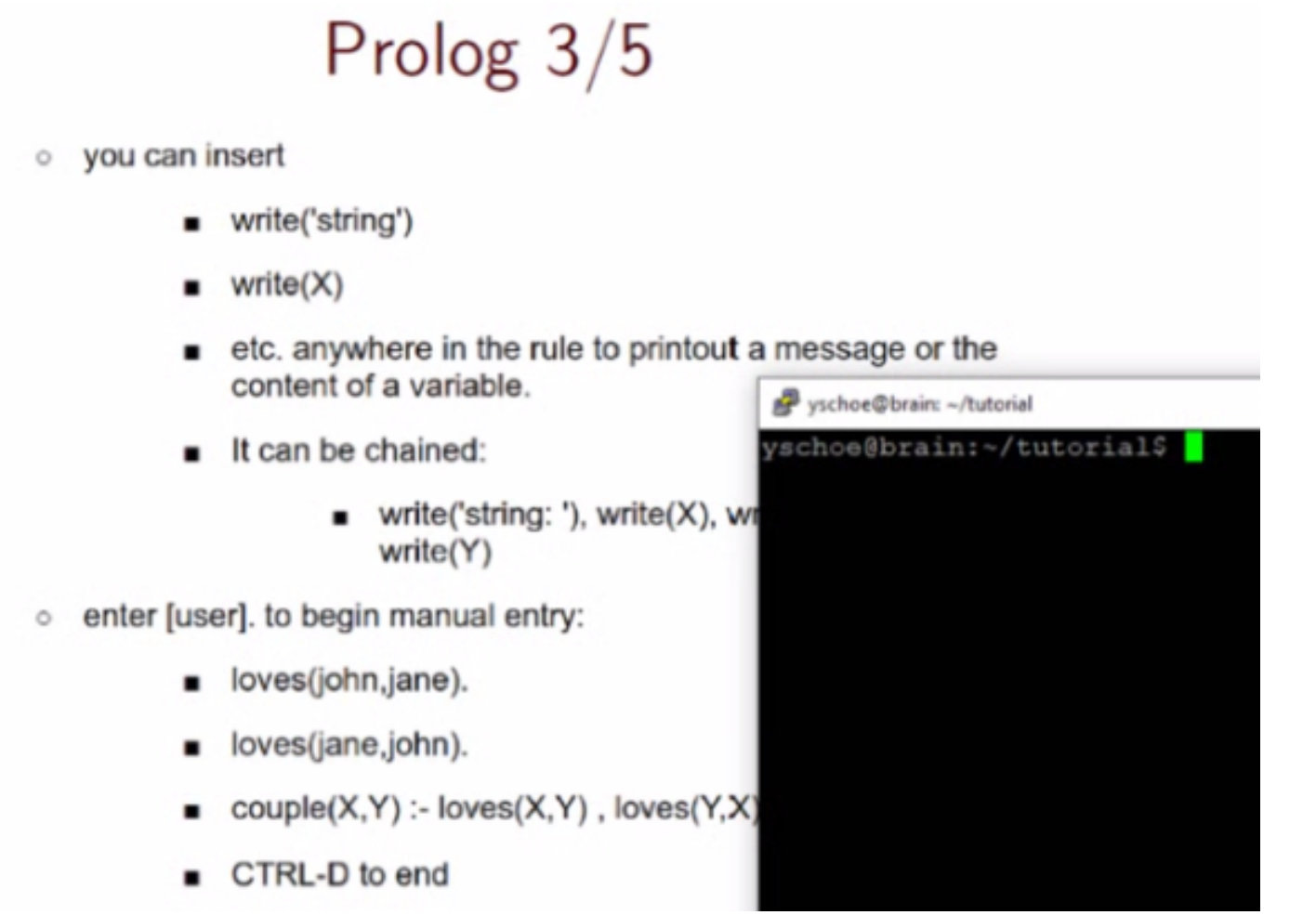
Prolog 3/5

- you can insert
  - write('string')
  - write(X)
  - etc. anywhere in the rule to printout a message or the content of a variable.
  - It can be chained:
    - write('string: '), write(X), write('string2: '), write(Y)
- enter [user]. to begin manual entry:
  - loves(john,jane).
  - loves(jane,john).
  - couple(X,Y) :- loves(X,Y) , loves(Y,X).
  - CTRL-D to end

We can also output a string or the value of a variable X using the write predicate. This write predicate can appear anywhere a predicate appears. And we can also link them up with "and," the comma sign, to write a more complex sentence.

As we have seen earlier, we can use the user command to start entering these facts and rules manually.

When we are done, we can press control-D to end and come back to the Prolog prompt.



Let us try this example. We run gprolog. Then enter the user command. Again, do not forget that period at the end, and now we are ready to enter our facts and rules: loves(john, jane), loves(jane, john). And the rule for "couple" is when X loves Y, and Y loves X. And again, this comma here means "and," and this, :-, means if.

So, this is true if that is the case. So, in implication notation, it would be loves(X, Y), and loves(Y, X), then couple(X, Y). Let us write that rule:

couple(X, Y) :- loves(X, Y) , loves(Y, X).

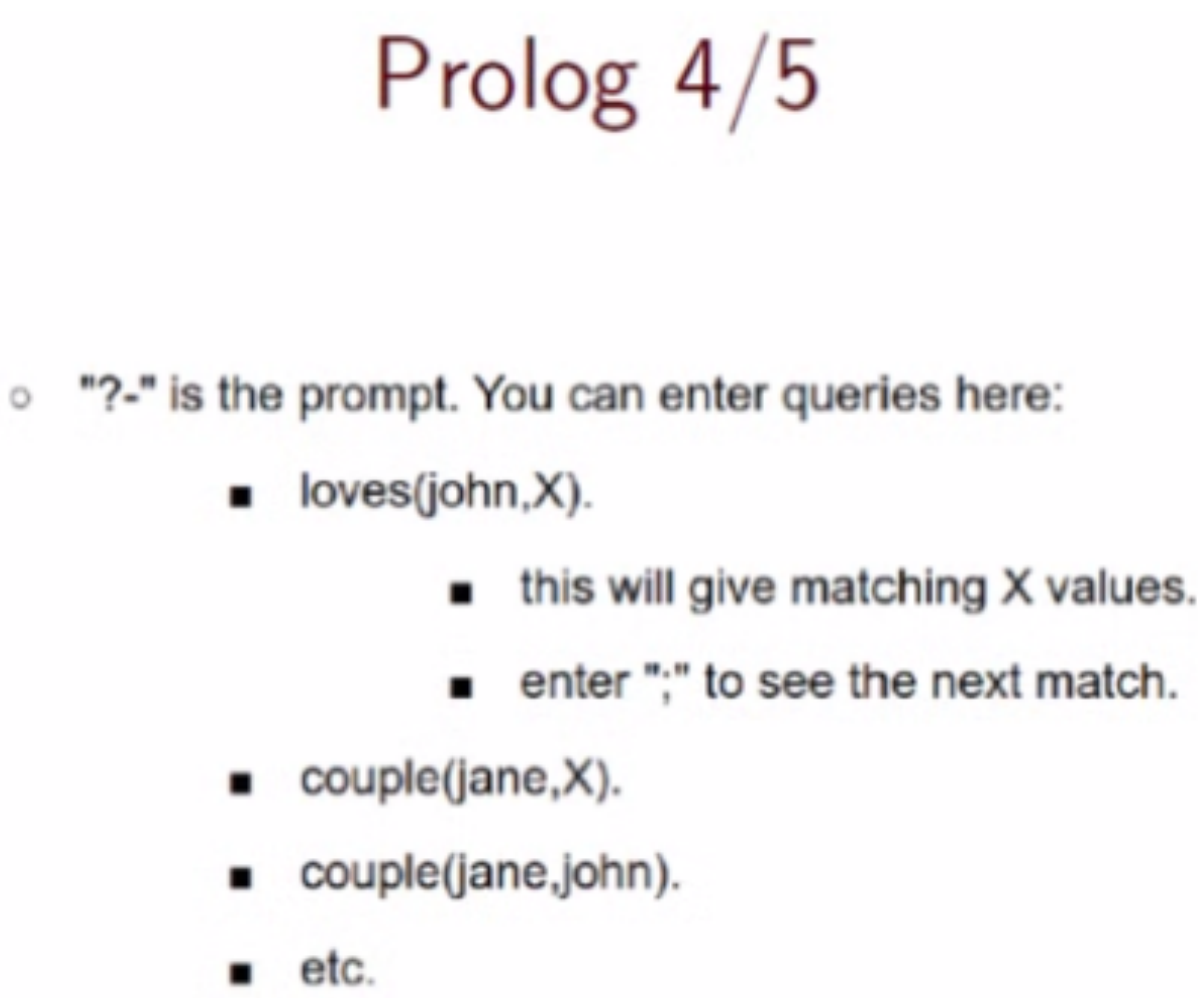
Again, another important thing to remember is that variables are represented as capital letters. Once we are done, we press control-D.

Here now, we can ask questions: loves(john, X). This is, whom does John Love? And it will give us the answer, Jane. And we can also check if John is a couple with someone. Let us first try whether John is a couple with Jane.

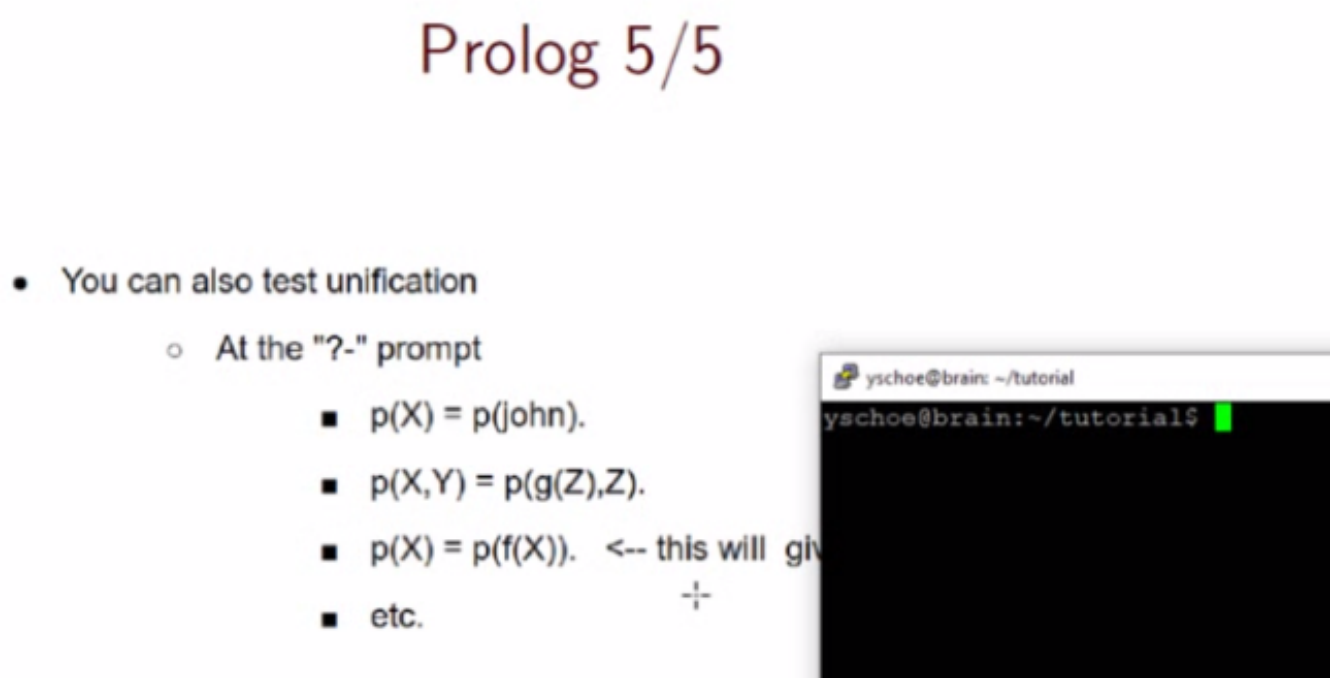
So, the answer is yes. And then we try the reverse. Is Jane a couple with John?

And, yes. Then again, by putting a variable in one of these arguments, we can ask a question, who is a couple with John?

Then, it will give us the answer, Jane. And again, when we are done with this Prolog session, press control-D.



This slide just summarizes what we did. We can ask questions. Who is loved by John and who is a couple with Jane, and whether Jane and John are couples.

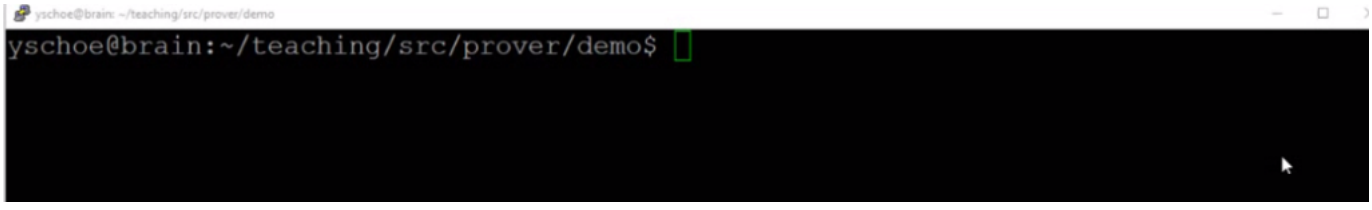


Another useful feature is we can use this to check whether two predicates would be unified. So, let us try this gprolog. Then here is the predicate with variable x, and with these two expressions, resolve. It will give us the substitution X should be John.

Let us try the next one, p(X, Y), with this unifier with p(g(Z), Z). And again, in this case, we replace X first g(Z) and Z with Y, which would be the unifier.

So, here is another interesting case: p(X) = p(f(X)).

So, in this case, we want to replace X with f(X), but X is included as an argument in this function, so it will give us some issues.



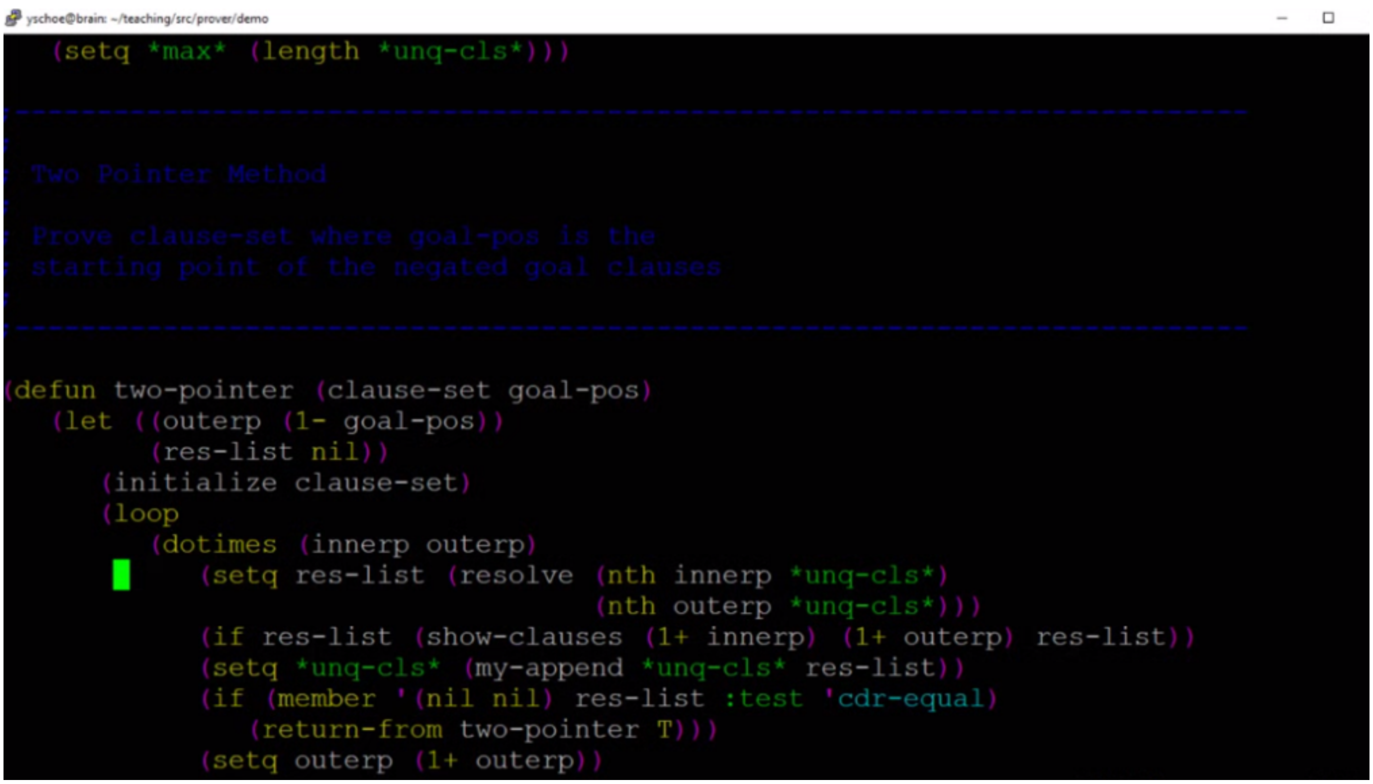
Finally, we have a full-blown resolution theorem prover, and I will show you a demo of this prover written in LISP.

For this, we will use the GNU Common LISP, and let me first show you the program.

Initially, I have all the definitions for the theorems.

Let us look at this Howling Hound theorem. So, it has eight different clauses in it, and each clause is separated into positive literals and negative literals. So, in the second clause, there are no positive literals, and all these are negative literals. And we can see we are using this weird LISP notation. So, have x y would be shown in this kind of format.

And there are these two algorithms that I am going to test. The first is the two-pointer or the lever saturation method.



So, here is the program, and that is it. Of course, there are lots of these calls.

And the next is unit preference.

```
        (list i (- j 2)))))

; temporary global variable to return clause numbers
(defvar *pair*)

;-----------------------------------------------------------------------
;
; Unit Preference
;
;-----------------------------------------------------------------------

(defun unit-preference (clause-set)
  (let ((ordered-clauses nil)
        (res-lst nil))
    (initialize clause-set)
    (while (setq res-lst (find-unit-resolution *unq-cls*))
      (if res-lst
          (show-clauses (first *pair*) (second *pair*) res-lst))
      (setq *unq-cls* (my-append *unq-cls* res-lst))
      (if (member '(nil nil) res-lst :test 'cdr-equal)
          (return-from unit-preference T)))
    'failure))

; find resolvent using shortest clauses
                                                    444,8         77%
```

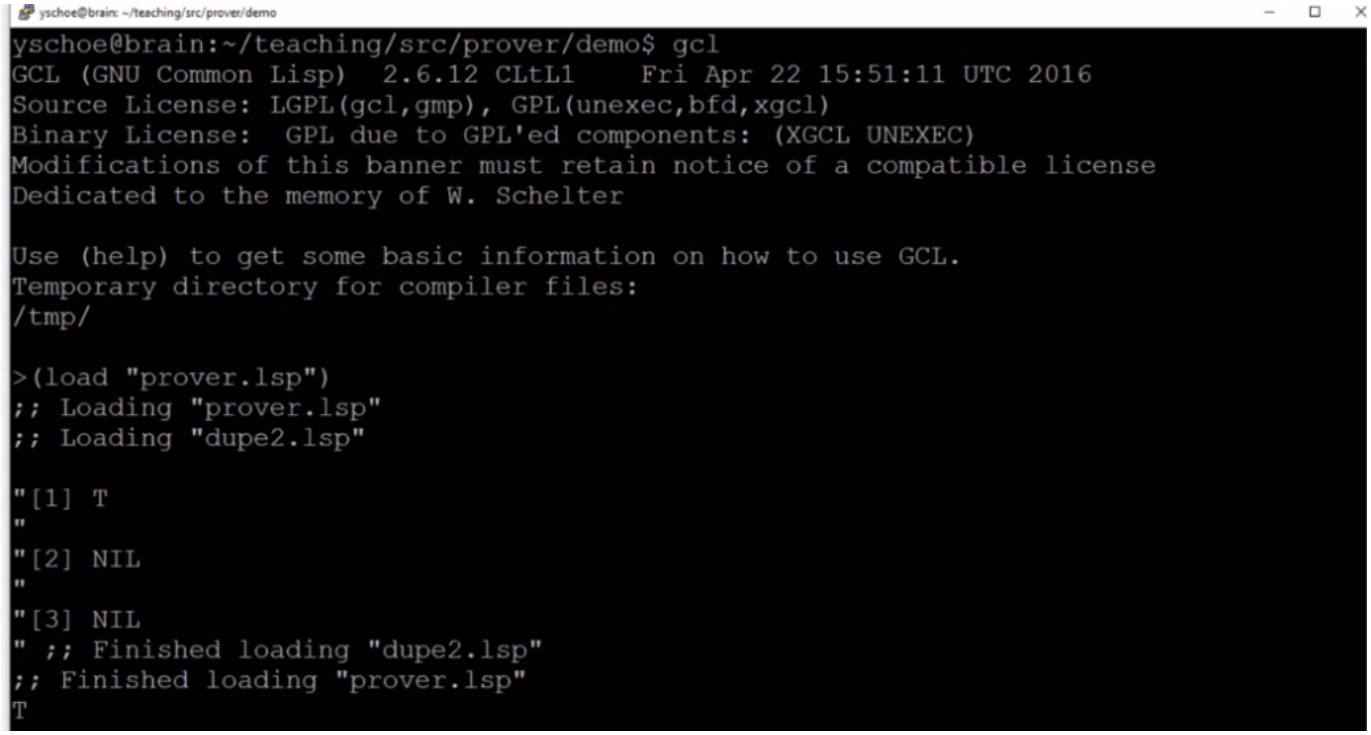
Then there is the full function. Of course, there are lots of other functions being called within this function.

```
yschoe@brain:~/teaching/src/prover/demo
yschoe@brain:~/teaching/src/prover/demo$ vi prover.lsp
yschoe@brain:~/teaching/src/prover/demo$
```

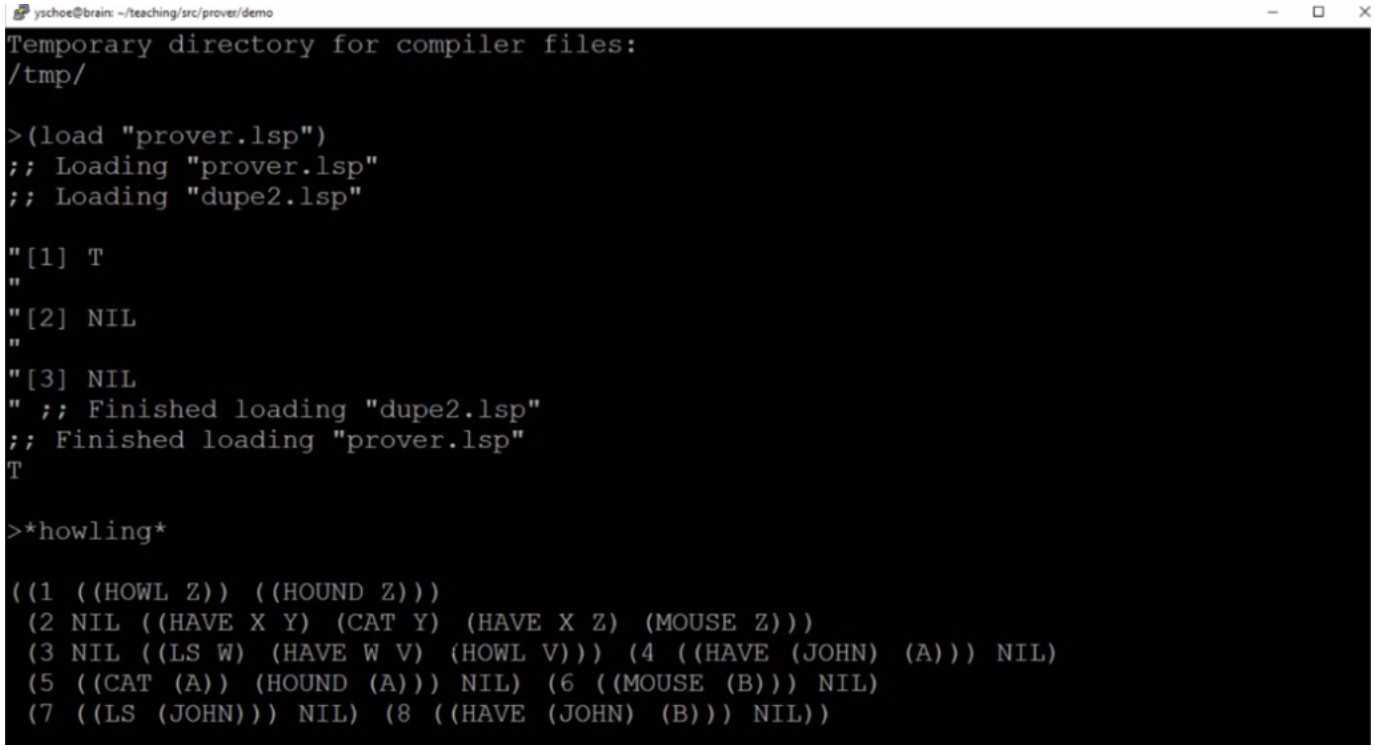
Now let us start gcl. That is the GNU Common LISP command.

```
yschoe@brain:~/teaching/src/prover/demo
yschoe@brain:~/teaching/src/prover/demo$ vi prover.lsp
yschoe@brain:~/teaching/src/prover/demo$ gcl
GCL (GNU Common Lisp)  2.6.12 CLtL1    Fri Apr 22 15:51:11 UTC 2016
Source License: LGPL(gcl,gmp), GPL(unexec,bfd,xgcl)
Binary License:  GPL due to GPL'ed components: (XGCL UNEXEC)
Modifications of this banner must retain notice of a compatible license
Dedicated to the memory of W. Schelter

Use (help) to get some basic information on how to use GCL.
Temporary directory for compiler files:
/tmp/

>
```

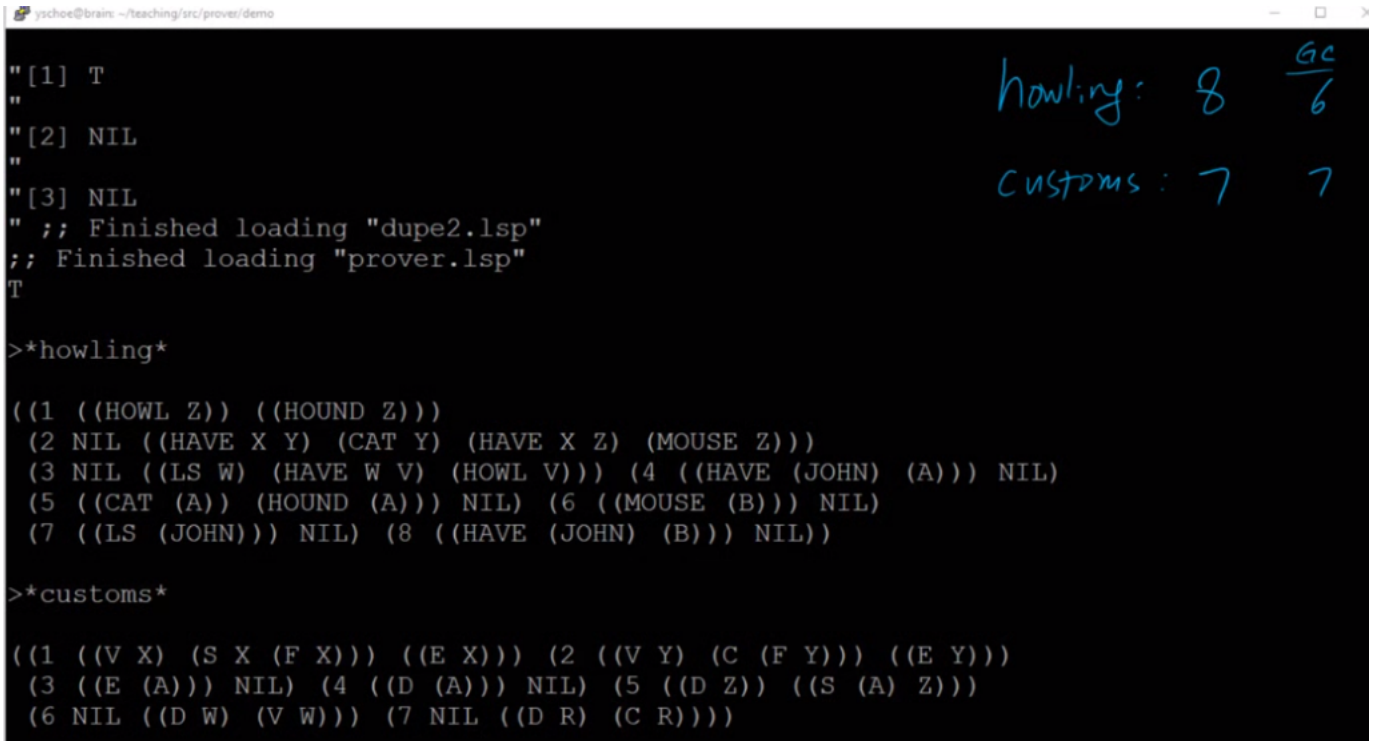
And here is the LISP prompt, and I load the file.



That is successful. And here, I can check the theorems.

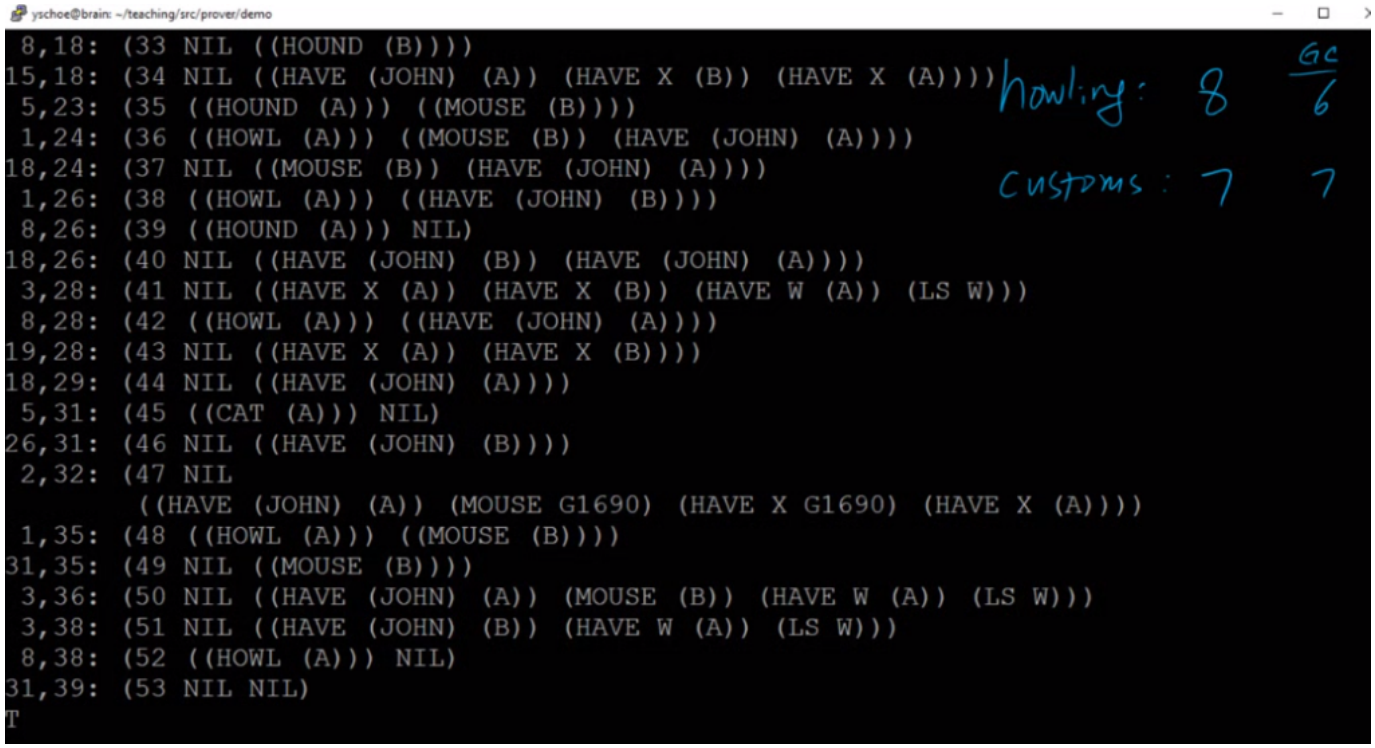


So, here is the first theorem: Howling Hound.



And here is the second theorem, the customs officers' one.

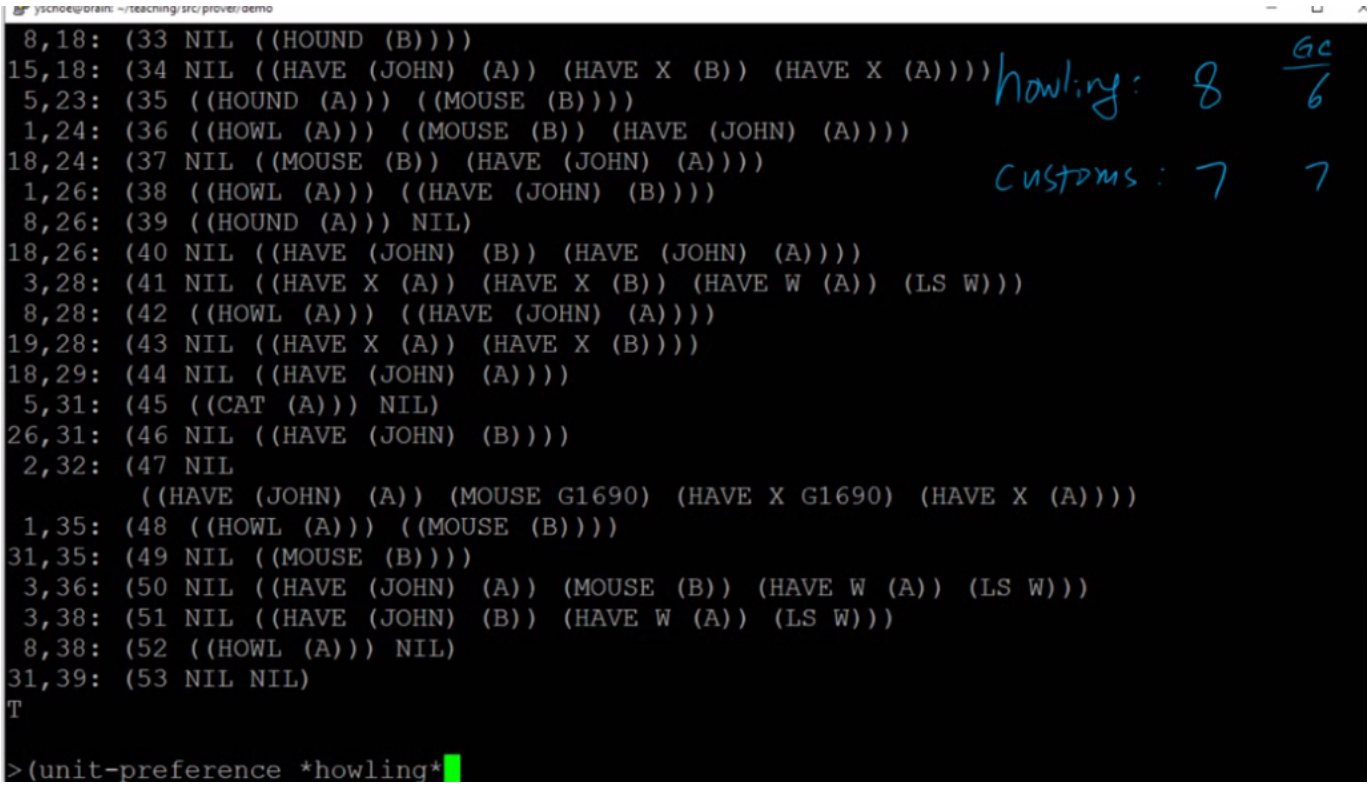
Now remember that the Howling Hound theorem had eight clauses to begin with, and the customs officials theorem had seven clauses to begin with.



Also, note that the goal clause for the Howling Hounds theorem starts at clause number six. So, let us write it down here. And for the customs officials' theorem, the goal clause is seven.

We will need this to use the set of support strategies.

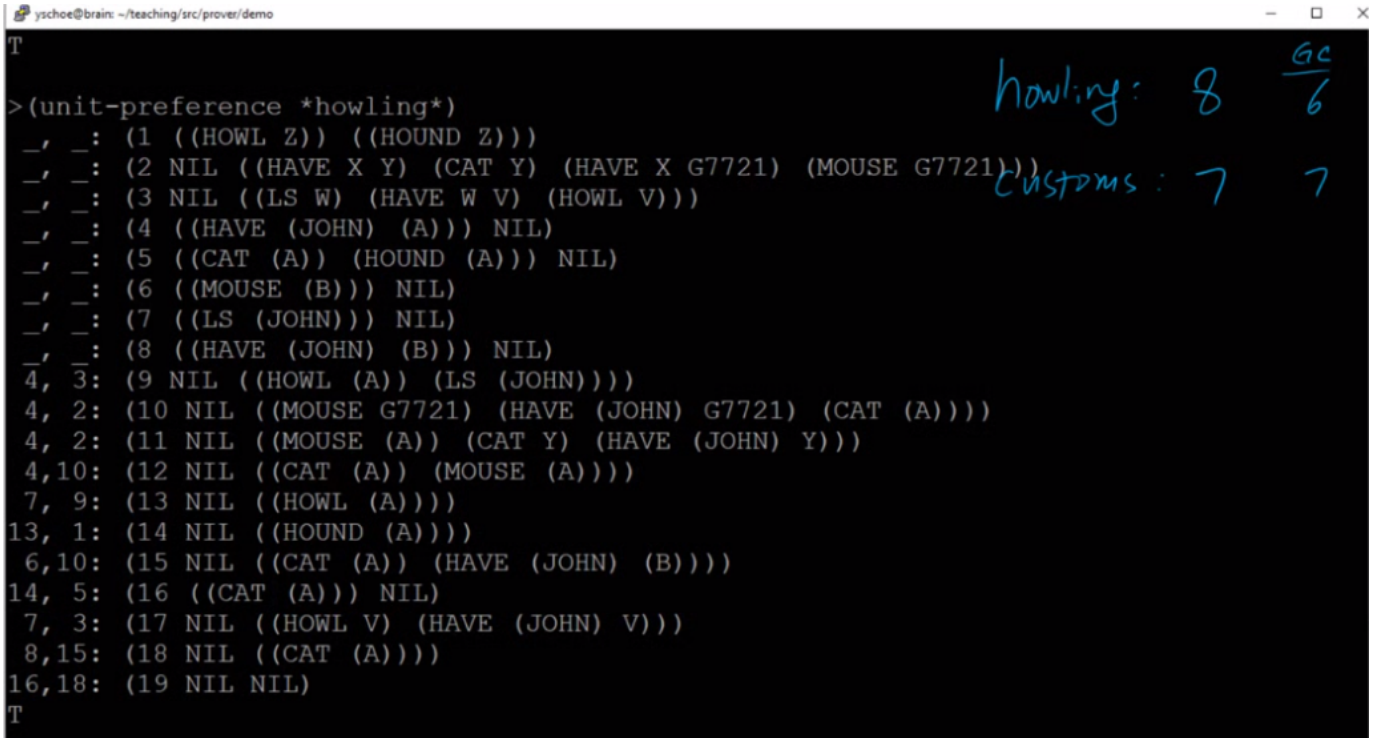
Now, let us try the two-pointer method with the set of support strategies, so Howling. Then we put in the goal clause number six.



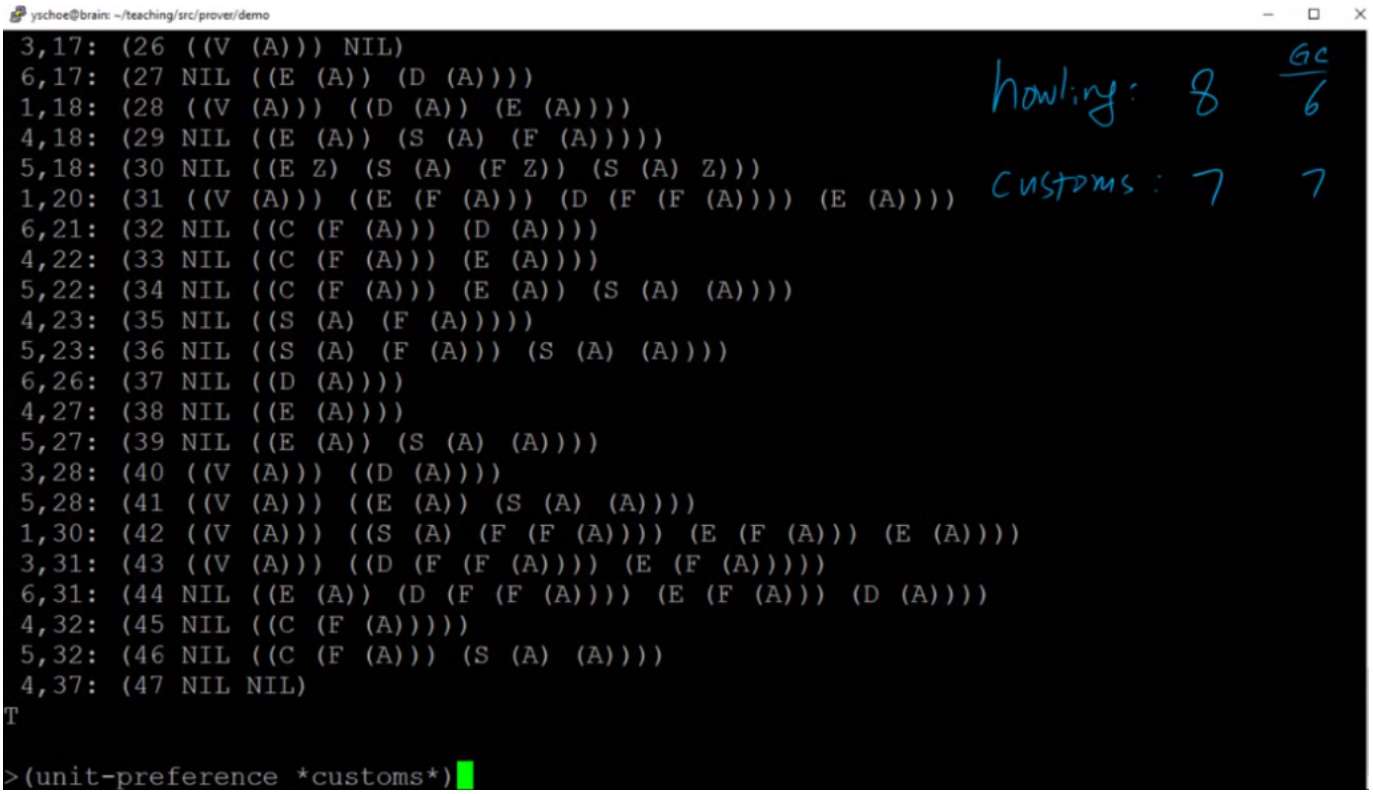
So this went through multiple resolution steps. In the end, in clause 53, the theorem has been proven.

If I recall correctly, when I was actually going through this proof, it only took five or six steps.

As we have discussed in the earlier lectures, this is what is expected because it will generate a lot of these clauses that are redundant and not needed. Furthermore, it is very difficult to understand what is going on by looking through these steps.



Next, let us try unit preference on the Howling Hound theorem.

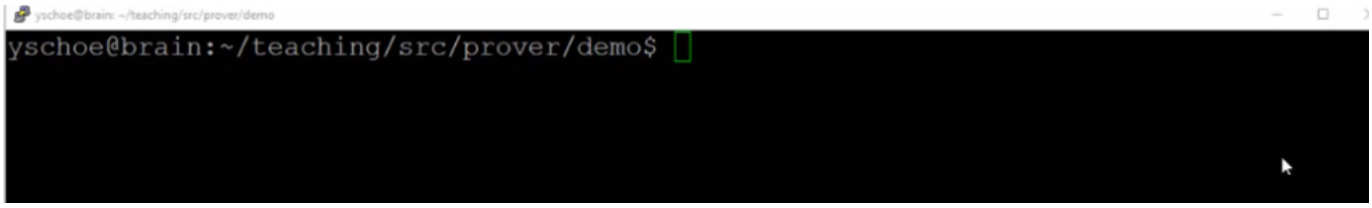


And in this case, it gave us the solution in 19 steps. So, compared to 53 in the two-pointer

method, this seems more efficient.

Next, let us try the customs officials' theorem.

Again, we are using the set of support strategies, so we need to put in the goal clause number, and after 47 steps, it generated a false. NIL NIL means that both the positive and negative literal are empty. Then let us try unit preference.



So, remember that it took 47 steps, and for this, it only took 18 steps. So again, the unit preference seems to be doing well.

With this, we are finally done with the topic of logic.

In the following module, we will look into planning.